

claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_e27060ad-65f\agent-a8d1e6399c4a45481.jsonl`  
- SHA-256: `71e047e5895f8cfc56c1cdf6f5a882b2c11ed0a994c3c965b5600c9375c8a219`  
- Source modified: `2026-06-19T16:41:23+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_e27060ad-65f`  
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`
```

Transcript

1. user (2026-06-19T16:40:08.115Z)

You are adversarially reviewing a technical claim in a proposed PR. Verify against REAL code in the SIBLING repo. Quote exact lines.

The sibling repo is at C:\Users\avidu\Projects\sports-data-collector (also an additional working dir). The file is src/parsers/cvat/rally_cvat.py.

```
PR CLAIM C ? "Graceful Degradation on Incomplete CSV Validation (rally_cvat.py)":  
> parse_cvat_boxes searches zips for an .xml ending in 'annotations.xml' or, failing that, ANY .xml.  
> In load_canonical_csv(), safe_int() catches malformed rally_number fields. If a row is entirely malformed (but doesn't break CSV parsing) it just continues silently. A severely malformed CSV might silently drop half its rallies without failing the import gate. Logging a warning for skipped rows would improve observability.  
The PLAN proposes: "Introduce logger.warning before the continue statement in load_canonical_csv to surface skipped rows during CVAT ingestion."
```

TASKS:

1. Read src/parsers/cvat/rally_cvat.py: parse_cvat_boxes (the xml-finding logic), load_canonical_csv, and safe_int. Quote the exact code. Is the description accurate?
2. Is the "silent continue" real? Is there ANY existing logging/warning around skipped rows? Is there already a logger in the module?
3. Read tests/test_cvat_robustness.py and tests/test_cvat_parser.py. What malformed-input behavior is already tested/locked in? Does any test assert silent-skip behavior that a warning would or wouldn't break?
4. Assess the proposed fix: is adding logger.warning before continue correct, safe, and trivially testable? Any reason it would NOT be a clean, low-risk improvement? Does the module already import logging?
5. Note: this fix is in a DIFFERENT repo (sports-data-collector), but the PR bundles it into the indexer's docs/plan. Flag the cross-repo coupling ? the indexer's convention is ONE PR per work item; a collector fix should land in the collector repo. Is the plan conflating two repos?

Return the structured verdict. claim_id = "C-cvat-parser".

2. assistant (2026-06-19T16:40:10.553Z)

I'll verify this claim against the real code in the sibling repo. Let me start by pulling the latest and reading the relevant files.

3. user (2026-06-19T16:40:13.929Z)

Already up to date.

```
---BRANCH---  
master  
---HEAD---
```

4. user (2026-06-19T16:40:14.402Z)

```

1      """Adapter: CVAT rally exports -> our canonical per-rally records.
2
3      Why this exists
4      -----
5      Our annotation brief (`docs/annotation/ROORA_BRIEF.md`) asks vendors for a
6      per-rally **CSV** that imports directly via `curate import-local`. A vendor may
7      instead do the work in CVAT and hand back the *CVAT* export: per-frame bounding
8      boxes, with the rally fields (`rally_number`, `shots_count`,
9      `ending_reason` ...) carried as CVAT **object attributes** on each box. The box
10     geometry is irrelevant to us ? it is just a carrier the rater attached the rally
11     metadata to.
12
13     This module is the ingestion-side adapter that absorbs that vendor format so the
14     modeling repo never has to. It does two things, both deliberately on *our* side
15     so we fix mechanical problems rather than renegotiating them:
16
17     1. **Recompute time from the frame range.** A rally's interval is taken from the
18         first/last *frame* a box for that rally appears on:
19         `seconds = frame / fps` at the video's true fps. We **ignore** any
20         `start_time`/`end_time` attributes the vendor wrote ? a CVAT job that
21         samples every `step` frames and assumes 30 fps yields systematically wrong
22         times (we have seen `start_time == frame / (step * 30)`, i.e. off by the
23         sampling step at the wrong fps). The frame index is reliable; the vendor's
24         seconds are not.
25
26     2. **Surface ? never silently fix ? content defects.** Overlapping rallies,
27         zero-duration rallies, ending_reason values outside the agreed set, and
28         too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
29         rally boundary would poison ground truth, so that stays a human/vendor call.
30
31     Supported input: the single-file CVAT XML export, in either **image mode**
32     (`<image><box>...`) or **track/interpolation mode** (`<track><box frame=...>`).
33     The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
34     later if a vendor sends only those.
35     """
36     from __future__ import annotations
37
38     import math
39     import os
40     import re
41     import zipfile
42     import xml.etree.ElementTree as ET
43     from collections import Counter
44     from dataclasses import dataclass, field
45     from typing import Any, Dict, List, Optional, Tuple
46
47     from src.core.parsing import safe_int
48     from src.core.downloader import probe_resolution_fps
49
50     # The agreed cross-sport ending_reason vocabulary for net-separated sports.
51     # Kept here as a soft check only (a warning, never a hard failure) so this
52     # adapter does not depend on the annotation-spec wording.
53     DEFAULT_ALLOWED_REASONS = {
54         "winner",
55         "forced_error",
56         "unforced_error",
57         "service_fault",
58         "let",
59         "other",
60     }
61

```

```

62 # The observational outcome of the last shot ? orthogonal to ending_reason
63 # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
64 DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
65
66 # Which court the rally is on, for MULTI-COURT videos (the model's weak spot:
67 # background-court pickups are its top false positive). "primary" = the main
68 # court the camera is filming; "secondary" = any background court. Blank/absent
69 # for single-court footage. Soft check only.
70 DEFAULT_ALLOWED_COURTS = {"primary", "secondary"}
71
72 # Per-rally annotator confidence. "unsure" rallies are surfaced on the review
73 # sheet (not errors ? requests for a second look) and can be down-weighted in
74 # training. Blank/absent is treated as "sure".
75 DEFAULT_ALLOWED_CONFIDENCE = {"sure", "unsure"}
76
77 # CVAT attribute names we map onto canonical columns. Vendor label schemas
78 # drift ? suffixed names like "score_before (A-B)", and singular/plural or
79 # case/seperator variants like "shot_count" (the Roora deliveries) for
80 # "shots_count" ? so both these aliases and the incoming attribute names are
81 # folded through ``_fold_attr_name`` before matching.
82 _ATTR_ALIASES = {
83     "rally_number": ("rally_number",),
84     "shots_count": ("shots_count", "shot_count"),
85     "ending_reason": ("ending_reason",),
86     "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
(in/net/out)"),
87     "court": ("court", "court_id", "court (primary/secondary)"),
88     "confidence": ("confidence", "sure", "review", "confidence (sure/unsure)"),
89     "score_before": ("score_before", "score_before (a-b)"),
90     "score_after": ("score_after", "score_after (a-b)"),
91     "notes": ("notes", "note"),
92 }
93
94
95 def _fold_attr_name(name: str) -> str:
96     """Fold a CVAT attribute name to a tolerant matching key.
97
98     Case ("Shot_Count"), separators ("shot count", "shot-count"), parenthesized
99     hints ("score_before (A-B)") and singular/plural ("shot_count" vs
100     "shots_count") all fold to the same key, so a vendor's naming drift cannot
101     silently drop a column.
102     """
103     s = re.sub(r"\(.*?\)", " ", (name or "").lower())
104     tokens = [t for t in re.split(r"[s_\-]+", s) if t]
105     return "_".join(
106         t[:-1] if len(t) > 1 and t.endswith("s") and not t.endswith("ss") else t
107         for t in tokens)
108
109
110 _FOLDED_ALIASES = {canon: tuple(dict.fromkeys(_fold_attr_name(a) for a in aliases))
111                      for canon, aliases in _ATTR_ALIASES.items()}
112
113 CANONICAL_COLUMNS = [
114     "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
115     "shots_count", "ending_reason", "last_shot_outcome", "court", "confidence",
116     "score_before", "score_after", "notes",
117 ]
118
119
120 @dataclass
121 class RallyRecord:
122     """One rally recovered from the CVAT export, time re-derived from frames."""
123     rally_number: int
124     start_time: float          # seconds, = frame_min / fps
125     end_time: float            # seconds, = frame_max / fps (see padding note)

```

```

126     shots_count: Optional[int]
127     ending_reason: Optional[str]
128     last_shot_outcome: Optional[str]
129     score_before: Optional[str]
130     score_after: Optional[str]
131     notes: Optional[str]
132     frame_min: int
133     frame_max: int
134     n_boxes: int
135     court: Optional[str] = None          # 'primary' | 'secondary' (multi-court)
136     confidence: Optional[str] = None    # 'sure' | 'unsure'
137     vendor_start_time: Optional[str] = None  # the (discarded) attribute value
138     vendor_end_time: Optional[str] = None
139
140     @property
141     def duration(self) -> float:
142         return self.end_time - self.start_time
143
144
145     @dataclass
146     class Issue:
147         """A defect found in the delivery. ``blocker`` issues stop a clean import."""
148         severity: str  # 'blocker' | 'high' | 'medium' | 'low'
149         code: str
150         message: str
151         rally_number: Optional[int] = None
152
153
154     @dataclass
155     class ConversionResult:
156         records: List[RallyRecord]
157         issues: List[Issue]
158         fps: float
159         step: int
160         meta: Dict[str, Any] = field(default_factory=dict)
161
162     @property
163     def n_blockers(self) -> int:
164         return sum(1 for i in self.issues if i.severity == "blocker")
165
166
167     def _mmss(seconds: float) -> str:
168         """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
169         m = int(seconds // 60)
170         s = seconds - m * 60
171         return f"{m}:{s:06.3f}"
172
173
174     def _norm_lower(value: Optional[str]) -> Optional[str]:
175         """Lowercase+strip a controlled-vocabulary value; None/blank stays None.
176
177         Used for ``court`` / ``confidence`` so 'Primary', ' primary ', 'PRIMARY' all
178         land on the canonical 'primary' the vocab checks expect.
179         """
180         if value is None:
181             return None
182         s = value.strip().lower()
183         return s or None
184
185
186     def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
187         """Look up a canonical field in a folded-key attrs dict (see _fold_attr_name)."""
188         for alias in _FOLDED_ALIASES[canonical]:
189             if alias in attrs and attrs[alias] != "":
190                 return attrs[alias]

```

```

191         return None
192
193
194     def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
195         """Parse a CVAT XML export into (meta, boxes).
196
197         ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
198         entry per visible box, from either image-mode or track-mode exports. Attr
199         names are folded via ``_fold_attr_name`` (lowercase, separators unified,
200         parenthesized hints dropped, plural-tolerant), not the raw vendor spelling.
201         ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
202         """
203         if not os.path.exists(xml_path):
204             raise FileNotFoundError(f"CVAT export not found: {xml_path}")
205
206         # Accept the standard CVAT delivery artifact (a .zip containing annotations.xml) as
207         # well as a bare .xml. Corrupt/unparseable XML is surfaced as a ValueError (a clean
208         # gate failure) rather than an uncaught ParseError traceback.
209         try:
210             if zipfile.is_zipfile(xml_path):
211                 with zipfile.ZipFile(xml_path) as zf:
212                     names = ([n for n in zf.namelist() if n.endswith("annotations.xml")]
213                             or [n for n in zf.namelist() if n.endswith(".xml")])
214                     if not names:
215                         raise ValueError(f"CVAT zip contains no annotations.xml:
{xml_path}")
216                     root = ET.fromstring(zf.read(names[0]))
217             else:
218                 root = ET.parse(xml_path).getroot()
219         except ET.ParseError as e:
220             raise ValueError(f"CVAT XML is corrupt/unparseable ({xml_path}): {e}")
221         meta: Dict[str, Any] = {}
222
223         job = root.find("./job")
224         if job is not None:
225             stop = job.findtext("stop_frame")
226             if stop is not None and stop.strip().isdigit():
227                 meta["stop_frame"] = int(stop.strip())
228             ff = job.findtext("frame_filter") or ""
229             if "step=" in ff:
230                 try:
231                     meta["step"] = int(ff.split("step=")[1].split()[0])
232                 except (ValueError, IndexError):
233                     pass
234
235         def _attrs(el: ET.Element) -> Dict[str, str]:
236             out: Dict[str, str] = {}
237             for a in el.findall("attribute"):
238                 out[_fold_attr_name(a.get("name"))] = (a.text or "").strip()
239             return out
240
241         boxes: List[Dict[str, Any]] = []
242
243         # Image mode: <image id=frame> ... <box>...</box>
244         for img in root.findall("image"):
245             try:
246                 frame = int(img.get("id"))
247             except (TypeError, ValueError):
248                 continue
249             for box in img.findall("box"):
250                 boxes.append({"frame": frame, "attrs": _attrs(box)})
251
252         # Track mode: <track> ... <box frame=N outside=0>...</box>
253         for track in root.findall("track"):
254             for box in track.findall("box"):

```

```

255         if box.get("outside") == "1":
256             continue # a track's terminating keyframe carries no real label
257         try:
258             frame = int(box.get("frame"))
259         except (TypeError, ValueError):
260             continue
261         boxes.append({"frame": frame, "attrs": _attrs(box)})
262
263     return meta, boxes
264
265
266 def _infer_step(frames: List[int]) -> int:
267     """Most common positive gap between consecutive distinct sampled frames."""
268     uniq = sorted(set(frames))
269     gaps = [b - a for a, b in zip(uniq, uniq[1:]) if b - a > 0]
270     if not gaps:
271         return 1
272     return Counter(gaps).most_common(1)[0][0]
273
274
275 def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
276     """Group boxes by rally_number and derive each rally's frame range -> seconds."""
277     if fps is None or fps <= 0:
278         raise ValueError("fps must be a positive number to convert frames to seconds")
279
280     by_rally: Dict[int, List[Dict[str, Any]]] = {}
281     for b in boxes:
282         rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
283         if rn is None:
284             continue
285         by_rally.setdefault(rn, []).append(b)
286
287     records: List[RallyRecord] = []
288     for rn in sorted(by_rally):
289         items = by_rally[rn]
290         frames = sorted(b["frame"] for b in items)
291         attrs = items[0]["attrs"]
292         fmin, fmax = frames[0], frames[-1]
293         records.append(RallyRecord(
294             rally_number=rn,
295             start_time=fmin / fps,
296             end_time=fmax / fps,
297             shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
298             ending_reason=_attr_lookup(attrs, "ending_reason"),
299             last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
300             court=_norm_lower(_attr_lookup(attrs, "court")),
301             confidence=_norm_lower(_attr_lookup(attrs, "confidence")),
302             score_before=_attr_lookup(attrs, "score_before"),
303             score_after=_attr_lookup(attrs, "score_after"),
304             notes=_attr_lookup(attrs, "notes"),
305             frame_min=fmin,
306             frame_max=fmax,
307             n_boxes=len(frames),
308             vendor_start_time=attrs.get("start_time"),
309             vendor_end_time=attrs.get("end_time"),
310         ))
311     return records
312
313
314 def detect_issues(records: List[RallyRecord],
315                  stop_frame: Optional[int] = None,
316                  allowed_reasons: Optional[set] = None,
317                  allowed_outcomes: Optional[set] = None,
318                  max_gap_s: float = 15.0,
319                  fps: Optional[float] = None,

```

```

320         boundary_tolerance_s: float = 0.3) -> List[Issue]:
321     """Find the defects that block or degrade a clean ground-truth import.
322
323     Sampling density INSIDE a rally is deliberately not flagged ? sparse
324     in-rally boxes are fine, and highlight compilation pads clips anyway
325     (badminton-highlight-indexer compiler, config-driven 0.5s/0.5s). What IS
326     flagged (`boundary_grid_snap`, when `fps` is known) is every rally
327     BOUNDARY sitting on a common sampling grid coarser than the ±0.3s bar:
328     that means first/last boxes were snapped to sampled frames instead of the
329     exact serve-contact/dead frame, which structurally cannot meet the golden
330     quality bar (Brief §4) no matter how careful the rater was ? the defect
331     that invalidated the pickleball/table-tennis pilot (2026-06-10 review).
332     """
333     allowed = allowed_reasons if allowed_reasons is not None else
DEFAULT_ALLOWED_REASONS
334     allowed_out = allowed_outcomes if allowed_outcomes is not None else
DEFAULT_ALLOWED_OUTCOMES
335     issues: List[Issue] = []
336
337     ordered = sorted(records, key=lambda r: r.rally_number)
338     for i, r in enumerate(ordered):
339         if r.end_time <= r.start_time:
340             issues.append(Issue(
341                 "blocker", "zero_duration",
342                 f"Rally {r.rally_number}: end <= start "
343                 f"({frames {r.frame_min}-{r.frame_max}}); single-sample or stray box.",
344                 r.rally_number))
345
346         if r.ending_reason is not None and r.ending_reason not in allowed:
347             issues.append(Issue(
348                 "medium", "ending_reason_vocab",
349                 f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
350                 f"not in the agreed set {sorted(allowed)}.",
351                 r.rally_number))
352
353         if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
354             issues.append(Issue(
355                 "low", "outcome_vocab",
356                 f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
357                 f"not in {sorted(allowed_out)}.",
358                 r.rally_number))
359
360         if r.court is not None and r.court not in DEFAULT_ALLOWED_COURTS:
361             issues.append(Issue(
362                 "low", "court_vocab",
363                 f"Rally {r.rally_number}: court '{r.court}' "
364                 f"not in {sorted(DEFAULT_ALLOWED_COURTS)}.",
365                 r.rally_number))
366
367         if r.confidence is not None and r.confidence not in DEFAULT_ALLOWED_CONFIDENCE:
368             issues.append(Issue(
369                 "low", "confidence_vocab",
370                 f"Rally {r.rally_number}: confidence '{r.confidence}' "
371                 f"not in {sorted(DEFAULT_ALLOWED_CONFIDENCE)}.",
372                 r.rally_number))
373         elif r.confidence == "unsure":
374             # Not an error ? the annotator asked for a second look; surface it on
375             # the review sheet so the human pass prioritises it.
376             issues.append(Issue(
377                 "low", "annotator_unsure",
378                 f"Rally {r.rally_number}: annotator marked confidence=unsure ? review.",
379                 r.rally_number))
380
381         # Inflated span: a long rally with implausibly few shots usually means a
382         # stray/duplicated box stretched the frame range past the real ending.

```

```

383         if r.shots_count and r.duration >= 5.0:
384             sps = r.shots_count / r.duration
385             if sps < 0.3:
386                 issues.append(Issue(
387                     "medium", "shot_density",
388                     f"Rally {r.rally_number}: {r.shots_count} shots over "
389                     f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
390                     f"may be inflated by a stray box.",
391                     r.rally_number))
392
393         if stop_frame is not None and r.frame_max > stop_frame:
394             issues.append(Issue(
395                 "high", "frame_out_of_range",
396                 f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
397                 f"last frame {stop_frame}.",
398                 r.rally_number))
399
400         if i > 0:
401             prev = ordered[i - 1]
402             if r.start_time < prev.end_time:
403                 issues.append(Issue(
404                     "blocker", "overlap",
405                     f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
406                     f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
407                     f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
408                     r.rally_number))
409             elif r.start_time - prev.end_time > max_gap_s:
410                 # A long unlabeled stretch between two rallies is a cheap, label-only
411                 # second opinion for the worst error (a missed rally). It won't catch a
412                 # rally MERGED into its neighbour's span (shot_density/inflated catches
413                 # that), but it does catch an omitted rally that left a visible hole.
414                 gap = r.start_time - prev.end_time
415                 issues.append(Issue(
416                     "medium", "possible_missed_rally",
417                     f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
418                     f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
419                     f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
420 gap.",
421                     r.rally_number))
422
423         grid = _detect_boundary_grid(ordered, fps, boundary_tolerance_s)
424         if grid is not None:
425             issues.append(grid)
426         return issues
427
428     def _detect_boundary_grid(records: List[RallyRecord], fps: Optional[float],
429                             tolerance_s: float) -> Optional[Issue]:
430         """Flag rally BOUNDARIES that all sit on one sampling grid (frame stride).
431
432         If every first/last box frame is congruent modulo a stride g (the gcd of
433         their pairwise differences), boundaries were snapped to sampled frames. A
434         stride coarser than the boundary tolerance makes the  $\pm$ tolerance bar
435         structurally unmeetable, so the delivery must be reworked ? a blocker.
436         Needs frame info (CVAT path) and fps; CSV-loaded records (frame_min == -1)
437         and small deliveries (< 3 rallies) are skipped to avoid false positives.
438         """
439         if not fps or fps <= 0:
440             return None
441         boundaries = [f for r in records for f in (r.frame_min, r.frame_max) if f >= 0]
442         if len(boundaries) < 6:
443             return None
444         base = boundaries[0]
445         g = 0
446         for f in boundaries[1:]:

```



```

447         g = math.gcd(g, abs(f - base))
448     if g < 2:          # frame-accurate (or no common grid) ? the healthy case
449         return None
450     stride_s = g / fps
451     if stride_s <= tolerance_s:
452         return None    # grid finer than the bar can still meet it
453     return Issue(
454         "blocker", "boundary_grid_snap",
455         f"All {len(boundaries)} rally boundaries sit on a {g}-frame grid "
456         f"({stride_s:.2f}s at {fps:.2f} fps) ? first/last boxes were snapped to "
457         f"sampling frames, so boundaries structurally cannot meet the "
458         f"±{tolerance_s:g}s bar (Brief §4). The first/last box of every rally "
459         f"must be placed on the exact serve-contact / dead frame (frame-step "
460         f"to it); in-between boxes may stay sparse.")
461
462
463 def convert(xml_path: str,
464            fps: Optional[float] = None,
465            video_path: Optional[str] = None,
466            step: Optional[int] = None,
467            allowed_reasons: Optional[set] = None) -> ConversionResult:
468     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
469
470     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
471     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
472     that silent guess is exactly the bug this adapter exists to undo.
473     """
474     meta, boxes = parse_cvat_boxes(xml_path)
475
476     if fps is None and video_path:
477         _, _, probed = probe_resolution_fps(video_path)
478         fps = probed
479     if fps is None or fps <= 0:
480         raise ValueError(
481             "Could not determine fps. Pass fps=... or a readable video_path so "
482             "frames can be converted to seconds (seconds = frame / fps).")
483
484     if step is None:
485         step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
486
487     records = build_records(boxes, fps)
488     issues = detect_issues(records,
489                           stop_frame=meta.get("stop_frame"),
490                           allowed_reasons=allowed_reasons,
491                           fps=fps)
492     return ConversionResult(records=records, issues=issues, fps=fps, step=step,
493                             meta=meta)
494
495 def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
496     """Write records as the canonical per-rally CSV that ``import-local`` reads."""
497     import csv
498     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
499     with open(out_path, "w", newline="", encoding="utf-8") as f:
500         w = csv.writer(f)
501         w.writerow(CANONICAL_COLUMNS)
502         for r in sorted(records, key=lambda x: x.rally_number):
503             w.writerow([
504                 r.rally_number,
505                 _mmss(r.start_time), _mmss(r.end_time),
506                 f"{r.start_time:.3f}", f"{r.end_time:.3f}",
507                 r.shots_count if r.shots_count is not None else "",
508                 r.ending_reason or "",
509                 r.last_shot_outcome or "",
510                 r.court or "", r.confidence or "",

```

```

511         r.score_before or "", r.score_after or "", r.notes or "",
512     ])
513
514
515 def render_issue_report(result: ConversionResult, source: str) -> str:
516     """A human-readable issue report (for review / sending back to the vendor)."""
517     lines = [
518         f"CVAT rally conversion report ? {source}",
519         f"  fps used          : {result.fps:.3f}",
520         f"  sampling step      : {result.step} frames (~{result.step / result.fps:.2f}s)",
521         f"  rallies parsed     : {len(result.records)}",
522         f"  issues              : {len(result.issues)} ({result.n_blockers} blocker)",
523         "",
524     ]
525     if not result.issues:
526         lines.append("No issues detected.")
527     else:
528         for sev in ("blocker", "high", "medium", "low"):
529             group = [i for i in result.issues if i.severity == sev]
530             if not group:
531                 continue
532             lines.append(f"[{sev.upper()}]")
533             for i in group:
534                 lines.append(f"  - ({i.code}) {i.message}")
535             lines.append("")
536     return "\n".join(lines)
537
538
539 # ----- #
540 # Validation reporting ? used by `curate validate-delivery`. #
541 # These turn a (records, issues) pair into the artifacts a human reviews: #
542 # a machine summary (JSON), a human report (md), and an annotatable sheet. #
543 # ----- #
544
545 def _parse_secs(value: Optional[str]) -> Optional[float]:
546     """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
547     s = (value or "").strip()
548     if not s:
549         return None
550     if ":" in s:
551         try:
552             total = 0.0
553             for part in s.split(":"):
554                 total = total * 60 + float(part)
555             return total
556         except ValueError:
557             return None
558     try:
559         return float(s)
560     except ValueError:
561         return None
562
563
564 def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
565     """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
566     just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
567     fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
568     CSV carries no frame info ? only the frame-derived checks become inapplicable.)
569     """
570     import csv as _csv
571     if not os.path.exists(csv_path):
572         raise FileNotFoundError(f"CSV not found: {csv_path}")
573     records: List[RallyRecord] = []
574     with open(csv_path, newline="", encoding="utf-8") as f:
575         reader = _csv.DictReader(f)

```

```

576         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
577 [])]
578     for raw in reader:
579         row = {(k or "").strip().lower(): v for k, v in raw.items()}
580         rn = safe_int(row.get("rally_number"), None)
581         if rn is None:
582             continue
583         start = _parse_secs(row.get("start_time"))
584         if start is None:
585             start = _parse_secs(row.get("start_mmss"))
586         end = _parse_secs(row.get("end_time"))
587         if end is None:
588             end = _parse_secs(row.get("end_mmss"))
589         if start is None or end is None:
590             continue
591         records.append(RallyRecord(
592             rally_number=rn, start_time=start, end_time=end,
593             shots_count=safe_int(row.get("shots_count"), None),
594             ending_reason=(row.get("ending_reason") or "").strip() or None,
595             last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
596             court=_norm_lower(row.get("court")),
597             confidence=_norm_lower(row.get("confidence")),
598             score_before=(row.get("score_before") or "").strip() or None,
599             score_after=(row.get("score_after") or "").strip() or None,
600             notes=(row.get("notes") or "").strip() or None,
601             frame_min=-1, frame_max=-1, n_boxes=0,
602         ))
603     return records
604
605 def build_summary(records: List[RallyRecord], issues: List[Issue],
606                  source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
607     """Machine-readable validation summary (JSON / CI)."""
608     by_sev = {s: [i for i in issues if i.severity == s]
609              for s in ("blocker", "high", "medium", "low")}
610     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
611     return {
612         "source": source,
613         "fps": fps,
614         "n_rallies": len(records),
615         "n_issues": len(issues),
616         "counts_by_severity": {s: len(v) for s, v in by_sev.items()},
617         "n_blockers": len(by_sev["blocker"]),
618         "passed": len(by_sev["blocker"]) == 0,
619         "flagged_rallies": flagged,
620         "issues": [{"severity": i.severity, "code": i.code,
621                    "rally_number": i.rally_number, "message": i.message} for i in
622 issues],
623     }
624
625 def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str)
626 -> str:
627     """Human validation report: scoreboard + issues by severity + rallies to review."""
628     n = len(records)
629     by_sev = {s: [i for i in issues if i.severity == s]
630              for s in ("blocker", "high", "medium", "low")}
631     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
632     clean = n - len(flagged)
633     lines = [
634         f"# Delivery validation ? {source}",
635         "",
636         f"- rallies: **{n}**",
637         f"- issues: **{len(issues)}** (blocker {len(by_sev['blocker'])}, "
638         f"high {len(by_sev['high'])}, medium {len(by_sev['medium'])}, low

```

```

{len(by_sev['low'])}]",
638         f"- structurally-clean rallies: **{clean}/{n}**"
639         + (f" ({100 * clean // n}%) " if n else ""),
640         f"- gate: **{'PASS' if not by_sev['blocker'] else 'BLOCKED'}**",
641         "",
642     ]
643     if issues:
644         for sev in ("blocker", "high", "medium", "low"):
645             grp = by_sev[sev]
646             if not grp:
647                 continue
648             lines.append(f"## {sev.upper()}")
649             lines += [f"- ({i.code}) {i.message}" for i in grp]
650             lines.append("")
651     else:
652         lines.append("No issues detected. ?\n")
653     if flagged:
654         lines.append(f"## Rallies to review: {flagged}")
655     return "\n".join(lines)
656
657
658 def write_review_csv(records: List[RallyRecord], issues: List[Issue], out_path: str) ->
None:
659     """Auto-generate the annotatable manual-verification sheet (blank `true_` cols)."""
660     import csv
661     flags: Dict[int, List[str]] = {}
662     for i in issues:
663         if i.rally_number is not None:
664             flags.setdefault(i.rally_number, []).append(i.code)
665     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
666     cols = ["rally_number", "start_mmss", "end_mmss", "dur_s", "shots_count",
667            "ending_reason", "last_shot_outcome", "auto_flags",
668            "VERIFIED(Y/N)", "true_start_mmss", "true_end_mmss",
669            "true_ending_reason", "true_last_shot_outcome", "reviewer_comment"]
670     with open(out_path, "w", newline="", encoding="utf-8") as f:
671         w = csv.writer(f)
672         w.writerow(cols)
673         for r in sorted(records, key=lambda x: x.rally_number):
674             w.writerow([
675                 r.rally_number, _mmss(r.start_time), _mmss(r.end_time),
676                 f"{r.duration:.1f}", r.shots_count if r.shots_count is not None else "",
677                 r.ending_reason or "", r.last_shot_outcome or "",
678                 ";".join(flags.get(r.rally_number, [])),
679                 "", "", "", "", "", "",
680             ])
681

```

5. assistant (2026-06-19T16:40:16.362Z)

Now let me read the safe_int helper and the test files.

6. user (2026-06-19T16:40:17.373Z)

```

1     """Shared parsing helpers used by the CLI importer and dataset parsers.
2
3     Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
4     ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
5     """
6     from typing import Any, Optional
7
8
9     def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10         """Parse an int from a possibly-empty/float/None value; return default on
failure."""
11         if value is None:

```

```

12         return default
13     s = str(value).strip()
14     if not s:
15         return default
16     try:
17         return int(float(s))
18     except (TypeError, ValueError):
19         return default
20
21
22     def safe_float_required(value: Any) -> float:
23         """Parse a float; raise ValueError if missing/empty (caller treats it as
24         required)."""
25         if value is None:
26             raise ValueError("missing required numeric value")
27         s = str(value).strip()
28         if not s:
29             raise ValueError("missing required numeric value")
30         return float(s)
31
32     def parse_time_string(time_str: Any) -> float:
33         """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
34         blank/invalid."""
35         if not time_str:
36             return 0.0
37         s = str(time_str).strip()
38         if ":" in s:
39             parts = s.split(":")
40             try:
41                 if len(parts) == 3:
42                     return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
43                 if len(parts) == 2:
44                     return float(parts[0]) * 60 + float(parts[1])
45             except ValueError:
46                 return 0.0
47         try:
48             return float(s)
49         except ValueError:
50             return 0.0

```

7. user (2026-06-19T16:40:17.818Z)

```

1     """validate-delivery / convert-cvat must FAIL (not silently PASS or traceback) on a CVAT
2     export that is empty, corrupt, or zipped.
3
4     Regression for: an empty CVAT (0 rallies) printed GATE: PASS; corrupt XML raised an
5     uncaught ParseError; the standard .zip delivery artifact was unsupported.
6     """
7     import argparse
8     import zipfile
9
10    import pytest
11
12    from src.parsers.cvat.rally_cvat import parse_cvat_boxes, convert
13    from src.cli.curate import SportsDataCLI
14
15
16    def _box(frame, attrs):
17        rows = "".join(f'<attribute name="{k}">{v}</attribute>' for k, v in attrs.items())
18        return (f'<image id="{frame}" name="f{frame:06d}" width="1920" height="1080">'
19                f'<box label="rally" xtl="1" ytl="1" xbr="2" ybr="2">{rows}</box></image>')
20
21

```

```

22     def _xml(body):
23         return ('<?xml version="1.0" encoding="utf-8"?><annotations><version>1.1</version>'
24
25         '<meta><job><stop_frame>1000</stop_frame><frame_filter>step=30</frame_filter>'
26         f'</job></meta>{body}</annotations>')
27
28     def _write(path, body):
29         path.write_text(_xml(body), encoding="utf-8")
30         return str(path)
31
32
33     def test_empty_cvat_converts_to_zero_records(tmp_path):
34         # Boxes whose rally_number attribute was renamed -> skipped -> zero rallies.
35         p = _write(tmp_path / "empty.xml", _box(30, {"wrong_name": "1"}))
36         result = convert(p, fps=30.0)
37         assert result.records == []
38
39
40     def test_corrupt_xml_raises_valueerror_not_parseerror(tmp_path):
41         p = tmp_path / "bad.xml"
42         p.write_text("<annotations><image><box>", encoding="utf-8") # truncated
43         with pytest.raises(ValueError):
44             parse_cvat_boxes(str(p))
45
46
47     def test_zip_delivery_is_parsed(tmp_path):
48         inner = _xml(_box(30, {"rally_number": "1"}) + _box(60, {"rally_number": "1"}))
49         zpath = tmp_path / "delivery.zip"
50         with zipfile.ZipFile(zpath, "w") as zf:
51             zf.writestr("annotations.xml", inner)
52         meta, boxes = parse_cvat_boxes(str(zpath))
53         assert len(boxes) == 2
54
55
56     def test_validate_delivery_blocks_on_empty_cvat(tmp_path):
57         p = _write(tmp_path / "empty.xml", _box(30, {"wrong_name": "1"}))
58         cli = SportsDataCLI()
59         args = argparse.Namespace(
60             input=p, csv=None, video_id="d", fps=30.0, video_path=None,
61             out_dir=str(tmp_path),
62             )
63         with pytest.raises(SystemExit) as exc:
64             cli.validate_delivery(args)
65         assert exc.value.code == 2 # GATE: BLOCKED, not PASS

```

8. user (2026-06-19T16:40:18.165Z)

```

1     """Tests for the CVAT rally adapter (src/parsers/cvat/rally_cvat.py).
2
3     The adapter's whole reason for existing is to (a) recompute rally time from the
4     box FRAME RANGE at the true fps ? ignoring the vendor's wrong start/end_time
5     attributes ? and (b) report content defects rather than silently fixing them.
6     These tests pin both behaviours, plus the end-to-end hand-off to import-local.
7     """
8     import csv as _csv
9
10    from src.parsers.cvat.rally_cvat import (
11        RallyRecord, parse_cvat_boxes, build_records, detect_issues, convert,
12        write_canonical_csv, load_canonical_csv, build_summary, render_validation_md,
13        write_review_csv,
14    )
15    from src.cli.curate import SportsDataCLI
16

```

```

17
18 # --- helpers -----
19
20 def _img_box(frame, attrs):
21     rows = "".join(f'<attribute name="{k}">{v}</attribute>' for k, v in attrs.items())
22     return (f'<image id="{frame}" name="frame_{frame:06d}" width="1920" height="1080">'
23           f'<box label="rally" xtl="1" ytl="1" xbr="2" ybr="2">{rows}</box></image>')
24
25
26 def _write_cvat(path, images, step=30, stop_frame=1000):
27     body = "".join(images)
28     xml = (
29         '<?xml version="1.0" encoding="utf-8"?><annotations><version>1.1</version>'
30         f'<meta><job><stop_frame>{stop_frame}</stop_frame>'
31         f'<frame_filter>step={step}</frame_filter></job></meta>'
32         f'{body}</annotations>'
33     )
34     path.write_text(xml, encoding="utf-8")
35     return str(path)
36
37
38 def _rec(rn, start, end, shots=5, reason="winner", fmin=None, fmax=None, n=3,
outcome=None):
39     return RallyRecord(
40         rally_number=rn, start_time=start, end_time=end, shots_count=shots,
41         ending_reason=reason, last_shot_outcome=outcome,
42         score_before=None, score_after=None, notes=None,
43         frame_min=fmin if fmin is not None else int(start * 60),
44         frame_max=fmax if fmax is not None else int(end * 60), n_boxes=n)
45
46
47 def _grid_recs(boundary_frames, fps):
48     """Records whose (fmin, fmax) pairs are taken from boundary_frames in order."""
49     recs = []
50     for i in range(0, len(boundary_frames), 2):
51         fmin, fmax = boundary_frames[i], boundary_frames[i + 1]
52         recs.append(_rec(i // 2 + 1, fmin / fps, fmax / fps, fmin=fmin, fmax=fmax))
53     return recs
54
55
56 # --- boundary_grid_snap: the pickleball/TT pilot defect -----
57
58 def test_grid_snap_flags_30_frame_grid_as_blocker():
59     # The exact pilot shape: every boundary a multiple of 30 at 59.94 fps (~0.5s).
60     fps = 59.94006
61     recs = _grid_recs([60, 210, 420, 630, 900, 1230, 1500, 1950], fps)
62     issues = detect_issues(recs, fps=fps)
63     grid = [i for i in issues if i.code == "boundary_grid_snap"]
64     assert len(grid) == 1 and grid[0].severity == "blocker"
65     assert "30-frame grid" in grid[0].message
66
67
68 def test_grid_snap_detects_offset_grids_too():
69     # Boundaries at 30k + 7: still a 30-frame grid (gcd of differences), still snapped.
70     fps = 59.94006
71     recs = _grid_recs([67, 247, 427, 667, 907, 1207], fps)
72     issues = detect_issues(recs, fps=fps)
73     assert any(i.code == "boundary_grid_snap" for i in issues)
74
75
76 def test_grid_snap_silent_on_frame_accurate_boundaries():
77     fps = 59.94006
78     recs = _grid_recs([61, 233, 419, 671, 905, 1213], fps) # no common stride
79     issues = detect_issues(recs, fps=fps)
80     assert not any(i.code == "boundary_grid_snap" for i in issues)

```

```

81
82
83     def test_grid_snap_silent_when_grid_finer_than_the_bar():
84         # A 2-frame stride at 59.94 fps (~0.03s) can still meet  $\pm 0.3s$  ? not a defect.
85         fps = 59.94006
86         recs = _grid_recs([60, 242, 420, 662, 900, 1202], fps) # gcd of diffs = 2
87         issues = detect_issues(recs, fps=fps)
88         assert not any(i.code == "boundary_grid_snap" for i in issues)
89
90
91     def test_grid_snap_skipped_without_fps_or_frames_or_enough_rallies():
92         fps = 59.94006
93         gridded = _grid_recs([60, 240, 420, 660, 900, 1200], fps)
94         # No fps (the canonical-CSV validation path).
95         assert not any(i.code == "boundary_grid_snap" for i in detect_issues(gridded))
96         # CSV-loaded records carry no frames (frame_min == -1).
97         no_frames = [_rec(1, 1.0, 4.0, fmin=-1, fmax=-1), _rec(2, 6.0, 9.0, fmin=-1,
98             fmax=-1),
99             _rec(3, 11.0, 15.0, fmin=-1, fmax=-1)]
100         assert not any(i.code == "boundary_grid_snap" for i in detect_issues(no_frames,
101             fps=fps))
102         # Two rallies = 4 boundaries: too small a sample to infer a grid.
103         small = _grid_recs([60, 240, 420, 660], fps)
104         assert not any(i.code == "boundary_grid_snap" for i in detect_issues(small,
105             fps=fps))
106
107     def test_court_and_confidence_parsed_and_normalized(tmp_path):
108         xml = _write_cvat(tmp_path / "j.xml", [
109             _img_box(60, {"rally_number": "1", "shots_count": "8", "ending_reason":
110                 "winner",
111                 "court": "Primary", "confidence": "Sure"}),
112             _img_box(120, {"rally_number": "1"}),
113             _img_box(240, {"rally_number": "2", "shots_count": "4", "ending_reason":
114                 "winner",
115                 "court": "secondary", "confidence": "unsure"}),
116             _img_box(300, {"rally_number": "2"}),
117         ])
118         res = convert(xml, fps=60.0)
119         r1, r2 = res.records
120         # case/whitespace normalised to the controlled vocabulary
121         assert r1.court == "primary" and r1.confidence == "sure"
122         assert r2.court == "secondary" and r2.confidence == "unsure"
123
124     def test_court_confidence_round_trip_through_canonical_csv(tmp_path):
125         xml = _write_cvat(tmp_path / "j.xml", [
126             _img_box(60, {"rally_number": "1", "ending_reason": "winner",
127                 "court": "primary", "confidence": "sure"}),
128             _img_box(120, {"rally_number": "1"}),
129         ])
130         res = convert(xml, fps=60.0)
131         out = tmp_path / "canon.csv"
132         write_canonical_csv(res.records, str(out))
133         # header carries the new columns
134         header = out.read_text(encoding="utf-8").splitlines()[0]
135         assert "court" in header and "confidence" in header
136         # and they survive a reload (the CSV is the durable artifact)
137         reloaded = load_canonical_csv(str(out))
138         assert reloaded[0].court == "primary"
139         assert reloaded[0].confidence == "sure"
140
141     def test_unsure_confidence_flags_for_review_not_error():
142         recs = [_rec(1, 1.0, 4.0), _rec(2, 6.0, 9.0)]

```



```

141     recs[0].confidence = "unsure"
142     issues = detect_issues(recs)
143     unsure = [i for i in issues if i.code == "annotator_unsure"]
144     assert len(unsure) == 1 and unsure[0].rally_number == 1
145     assert unsure[0].severity == "low"           # a review request, never a blocker
146
147
148     def test_court_and_confidence_vocab_warnings():
149         recs = [_rec(1, 1.0, 4.0), _rec(2, 6.0, 9.0)]
150         recs[0].court = "court_5"                # off-vocab
151         recs[1].confidence = "maybe"           # off-vocab
152         codes = {i.code for i in detect_issues(recs)}
153         assert "court_vocab" in codes
154         assert "confidence_vocab" in codes
155
156
157     def test_grid_snap_blocks_the_convert_gate(tmp_path):
158         # End-to-end: an image-mode export with all boxes (hence all boundaries) on
159         # the 30-grid must come out of convert() carrying the blocker.
160         images = []
161         for rn, (f0, f1) in enumerate([(60, 240), (420, 660), (900, 1200)], start=1):
162             for f in range(f0, f1 + 1, 30):
163                 images.append(_img_box(f, {"rally_number": str(rn), "shots_count": "5",
164                                           "ending_reason": "winner"}))
165         xml = _write_cvat(tmp_path / "gridded.xml", images, step=30, stop_frame=2000)
166         res = convert(xml, fps=59.94006)
167         assert any(i.code == "boundary_grid_snap" and i.severity == "blocker"
168                   for i in res.issues)
169         assert res.n_blockers >= 1
170
171
172     # --- the core fix: time from frames, vendor times ignored -----
173
174     def test_time_recomputed_from_frames_not_attributes(tmp_path):
175         # rally 1 spans source frames 60..120; the vendor's start/end attributes are
176         # deliberately wrong (the frame/900 bug). At 60 fps the truth is 1.0..2.0s.
177         xml = _write_cvat(tmp_path / "j.xml", [
178             _img_box(60, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4",
179                           "shots_count": "6.0", "ending_reason": "winner",
180                           "score_before (A-B)": "0-0", "notes": "clean"}),
181             _img_box(90, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4"}),
182             _img_box(120, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4"}),
183         ])
184         res = convert(xml, fps=60.0)
185         assert len(res.records) == 1
186         r = res.records[0]
187         assert r.start_time == 1.0 and r.end_time == 2.0           # frames/fps, not 0.2/1.4
188         assert r.vendor_start_time == "0.2"                       # preserved for transparency
189         assert r.shots_count == 6
190         assert r.ending_reason == "winner"
191         assert r.score_before == "0-0"                            # "(A-B)" alias resolved
192         assert r.notes == "clean"
193
194
195     def test_attr_names_matched_tolerantly(tmp_path):
196         # Regression: the Roora deliveries named the attribute "shot_count"
197         # (singular); the strict alias table dropped it and every rally exported
198         # with an empty shots_count column. Vendor naming drift ? case, separators,
199         # parenthesized hints, singular/plural ? must fold onto the canonical columns.
200         xml = _write_cvat(tmp_path / "j.xml", [
201             _img_box(60, {"shot_count": "7", "rally_number": "1",
202                           "Ending Reason": "winner",
203                           "score_before(A-B)": "3-2"}),           # no space before "(A-B)"
204             _img_box(120, {"rally_number": "1"}),
205         ])

```

```

206     res = convert(xml, fps=60.0)
207     assert len(res.records) == 1
208     r = res.records[0]
209     assert r.shots_count == 7          # was silently None before the fold
210     assert r.ending_reason == "winner"
211     assert r.score_before == "3-2"
212
213
214 def test_step_and_fps_read_from_meta_and_arg(tmp_path):
215     xml = _write_cvat(tmp_path / "j.xml", [
216         _img_box(0, {"rally_number": "1", "shots_count": "3"}),
217         _img_box(30, {"rally_number": "1"}),
218     ], step=30)
219     res = convert(xml, fps=60.0)
220     assert res.step == 30
221     assert res.fps == 60.0
222
223
224 def test_fps_required_when_unavailable(tmp_path):
225     xml = _write_cvat(tmp_path / "j.xml", [_img_box(0, {"rally_number": "1"})])
226     try:
227         convert(xml) # no fps, no video_path
228     except ValueError as e:
229         assert "fps" in str(e).lower()
230     else:
231         raise AssertionError("expected ValueError when fps cannot be determined")
232
233
234 # --- track-mode parsing + outside filtering -----
235
236 def test_track_mode_and_outside_filtering(tmp_path):
237     xml = (
238         '<?xml version="1.0"?><annotations><meta><job>'
239         '<stop_frame>1000</stop_frame><frame_filter>step=30</frame_filter></job></meta>'
240         '<track id="0" label="rally">'
241         '<box frame="60" outside="0" xtl="1" ytl="1" xbr="2" ybr="2">'
242         '<attribute name="rally_number">1</attribute>'
243         '<attribute name="shots_count">4</attribute></box>'
244         '<box frame="120" outside="0" xtl="1" ytl="1" xbr="2" ybr="2">'
245         '<attribute name="rally_number">1</attribute></box>'
246         '<box frame="150" outside="1" xtl="1" ytl="1" xbr="2" ybr="2">' # terminator
247         '<attribute name="rally_number">1</attribute></box>'
248         '</track></annotations>'
249     )
250     p = tmp_path / "track.xml"
251     p.write_text(xml, encoding="utf-8")
252     meta, boxes = parse_cvat_boxes(str(p))
253     assert len(boxes) == 2          # the outside=1 terminator dropped
254     recs = build_records(boxes, fps=60.0)
255     assert recs[0].frame_max == 120 # not 150 (the terminator)
256     assert recs[0].end_time == 2.0
257
258
259 # --- issue detection: surfaced, not silently fixed -----
260
261 def test_overlap_is_a_blocker():
262     recs = [_rec(1, 1.0, 5.0), _rec(2, 4.0, 8.0)] # r2 starts before r1 ends
263     issues = detect_issues(recs)
264     overlaps = [i for i in issues if i.code == "overlap"]
265     assert len(overlaps) == 1 and overlaps[0].severity == "blocker"
266
267
268 def test_zero_duration_is_a_blocker():
269     recs = [_rec(1, 2.0, 2.0, fmin=120, fmax=120, n=1)]
270     issues = detect_issues(recs)

```

```

271         assert any(i.code == "zero_duration" and i.severity == "blocker" for i in issues)
272
273
274     def test_offvocab_ending_reason_flagged():
275         recs = [_rec(1, 1.0, 3.0, reason="into_net")]
276         issues = detect_issues(recs)
277         assert any(i.code == "ending_reason_vocab" for i in issues)
278         # the adopted forced/unforced terms are accepted, not flagged
279         ok = detect_issues(_rec(1, 1.0, 3.0, reason="forced_error"))
280         assert not any(i.code == "ending_reason_vocab" for i in ok)
281
282
283     def test_low_shot_density_flags_inflated_span():
284         recs = [_rec(1, 0.0, 16.0, shots=2)]          # 2 shots over 16s -> 0.12/s
285         issues = detect_issues(recs)
286         assert any(i.code == "shot_density" for i in issues)
287
288
289     def test_outcome_vocab_flagged():
290         bad = detect_issues(_rec(1, 1.0, 3.0, outcome="long"))
291         assert any(i.code == "outcome_vocab" for i in bad)
292         # the agreed outcomes are accepted, not flagged
293         ok = detect_issues(_rec(1, 1.0, 3.0, outcome="out"))
294         assert not any(i.code == "outcome_vocab" for i in ok)
295
296
297     # --- end-to-end: emitted CSV imports via the existing import-local parser ---
298
299     def test_canonical_csv_round_trips_through_import_local(tmp_path):
300         xml = _write_cvat(tmp_path / "j.xml", [
301             _img_box(60, {"rally_number": "1", "start_time": "0.2", "end_time": "1.4",
302                 "shots_count": "6", "ending_reason": "winner",
303                 "last_shot_outcome": "in"}),
304             _img_box(120, {"rally_number": "1"}),
305             _img_box(180, {"rally_number": "2", "shots_count": "4",
306                 "ending_reason": "forced_error", "last_shot_outcome": "out"}),
307             _img_box(240, {"rally_number": "2"}),
308         ])
309         res = convert(xml, fps=60.0)
310         assert res.n_blockers == 0
311         assert res.records[1].last_shot_outcome == "out"    # parsed from the box attribute
312         out = tmp_path / "canonical.csv"
313         write_canonical_csv(res.records, str(out))
314
315         # Re-read with the actual import-local parser (self is unused -> None is safe).
316         items = SportsDataCLI._parse_local_annotations(None, "badminton", "vid1", str(out))
317         assert len(items) == 2
318         assert items[0].start_time == 1.0 and items[0].end_time == 2.0    # not 0.2/1.4
319         assert items[1].ending_reason == "forced_error"
320         assert items[1].last_shot_outcome == "out"          # survives the round-trip into
the schema
321         assert items[1].start_time == 3.0 and items[1].end_time == 4.0
322
323
324     # --- validate-delivery: gap heuristic, CSV loading, report artifacts -----
325
326     def test_possible_missed_rally_gap_flagged():
327         # 20s of unlabeled time between two rallies -> flag (catches the 1:46-style miss)
328         recs = [_rec(1, 95.6, 98.0), _rec(2, 118.0, 119.6)]
329         issues = detect_issues(recs)
330         assert any(i.code == "possible_missed_rally" for i in issues)
331
332
333     def test_normal_gap_not_flagged():
334         recs = [_rec(1, 10.0, 14.0), _rec(2, 22.0, 26.0)]    # 8s gap -> fine

```

```

335     issues = detect_issues(recs)
336     assert not any(i.code == "possible_missed_rally" for i in issues)
337
338
339     def test_load_canonical_csv_decimal_and_mmss(tmp_path):
340         p = tmp_path / "c.csv"
341         p.write_text(
342             "rally_number,start_time,end_time,start_mmss,end_mmss,shots_count,ending_reason,
last_shot_outcome\n"
343             "1,3.0,21.0,,,15,unforced_error,out\n"
344             "2,,,0:29.000,0:32.000,2,winner,in\n", encoding="utf-8")
345         recs = load_canonical_csv(str(p))
346         assert len(recs) == 2
347         assert recs[0].start_time == 3.0 and recs[0].last_shot_outcome == "out"
348         assert recs[1].start_time == 29.0 and recs[1].end_time == 32.0 # mm:ss fallback
349
350
351     def test_load_canonical_csv_keeps_rally_starting_at_zero(tmp_path):
352         # Regression: a rally that legitimately starts at 0.0s must not be dropped.
353         # The old `start_time or start_mmss` fallback treated the falsy 0.0 as "missing"
354         # and skipped the row when there was no start_mmss column to fall back to.
355         p = tmp_path / "c.csv"
356         p.write_text(
357             "rally_number,start_time,end_time,shots_count,ending_reason,last_shot_outcome\n"
358             "1,0.0,4.0,8,winner,in\n"
359             "2,10.0,14.0,6,forced_error,out\n"
360             "3,20.0,24.0,5,unforced_error,out\n", encoding="utf-8")
361         recs = load_canonical_csv(str(p))
362         assert len(recs) == 3 # the zero-start row 1 is not lost
363         assert recs[0].start_time == 0.0 and recs[0].end_time == 4.0
364
365
366     def test_validation_artifacts(tmp_path):
367         recs = [_rec(1, 1.0, 5.0), _rec(2, 4.0, 8.0, reason="into_net")] # overlap + bad
vocab
368         issues = detect_issues(recs)
369         summary = build_summary(recs, issues, source="vid", fps=59.94)
370         assert summary["n_rallies"] == 2
371         assert summary["passed"] is False and summary["n_blockers"] >= 1
372         assert 2 in summary["flagged_rallies"]
373         md = render_validation_md(recs, issues, "vid")
374         assert "BLOCKED" in md and "## BLOCKER" in md
375         out = tmp_path / "review.csv"
376         write_review_csv(recs, issues, str(out))
377         rows = list(_csv.DictReader(open(out, encoding="utf-8")))
378         assert "VERIFIED(Y/N)" in rows[0] and "auto_flags" in rows[0]
379         assert any("overlap" in r["auto_flags"] for r in rows)
380

```

9. assistant (2026-06-19T16:40:21.877Z)

Now let me check whether the module imports logging anywhere, and whether there's an existing logger used in the codebase for this kind of pattern.

10. user (2026-06-19T16:40:23.434Z)

No matches found

11. user (2026-06-19T16:40:24.409Z)

No files found

12. assistant (2026-06-19T16:40:27.932Z)

The module has no logging import, and the entire `src/` tree of the collector repo uses no `logging`/`getLogger` at all. Let me confirm how warnings/observability are conventionally surfaced in this module and CLI ? the `Issue`/`print` pattern.

13. user (2026-06-19T16:40:29.517Z)

No matches found

14. user (2026-06-19T16:40:30.412Z)

```
433:         continue
447:         continue
452:         continue
461:         continue
470:         continue
521:         continue
579:         continue
588:         continue
593:         continue
868:         continue
964:     def validate_delivery(self, args):
976:         convert, load_canonical_csv, detect_issues,
999:         records = load_canonical_csv(args.csv)
1422:     cli.validate_delivery(args)
```

15. assistant (2026-06-19T16:40:33.502Z)

Let me look at how `validate\_delivery` consumes `load\_canonical\_csv` to understand the observability context ? whether skipped rows would surface elsewhere.

16. user (2026-06-19T16:40:34.150Z)

```
964     def validate_delivery(self, args):
965         """Run the full validation suite on a rally delivery and emit a report.
966
967         Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
968         CSV (--csv). Writes, into --out-dir:
969             <id>_report.md    human report (scoreboard + issues + rallies to review)
970             <id>_report.json  machine summary (for CI / a bulk-ingest loop)
971             <id>_review.csv   the annotatable manual-verification sheet
972         Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
973         """
974         import json
975         from src.parsers.cvat.rally_cvat import (
976             convert, load_canonical_csv, detect_issues,
977             build_summary, render_validation_md, write_review_csv,
978         )
979         vid = args.video_id or "delivery"
980         fps = None
981         step = None
982         stop_frame = None
983         if args.input:
984             try:
985                 result = convert(args.input, fps=args.fps, video_path=args.video_path)
986             except (FileNotFoundError, ValueError) as e:
987                 print(f"Error: {e}")
988                 sys.exit(1)
989             records, issues, fps = result.records, result.issues, result.fps
990             step = result.step
991             stop_frame = result.meta.get("stop_frame")
992             if not records:
993                 print(f"Error: the CVAT export parsed to ZERO rallies ({args.input}). "
994                       f"The wrong job was exported or the rally_number attribute was
renamed. "
```

```

995             f"GATE: BLOCKED.")
996             sys.exit(2)
997     elif args.csv:
998         try:
999             records = load_canonical_csv(args.csv)
1000         except FileNotFoundError as e:
1001             print(f"Error: {e}")
1002             sys.exit(1)
1003         if not records:
1004             print(f"Error: no usable rally rows in {args.csv}")
1005             sys.exit(1)
1006         issues = detect_issues(records)
1007     else:
1008         print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
1009         sys.exit(1)
1010
1011     os.makedirs(args.out_dir, exist_ok=True)
1012     summary = build_summary(records, issues, source=vid, fps=fps)
1013     md = render_validation_md(records, issues, source=vid)
1014     json_path = os.path.join(args.out_dir, f"{vid}_report.json")
1015     md_path = os.path.join(args.out_dir, f"{vid}_report.md")
1016     review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
1017     with open(json_path, "w", encoding="utf-8") as f:
1018         json.dump(summary, f, indent=2)
1019     with open(md_path, "w", encoding="utf-8") as f:
1020         f.write(md)
1021     write_review_csv(records, issues, review_path)
1022
1023     # Observability report (envelope JSON + technical md + customer summary),
1024     # colocated with the existing artifacts. Never raises.
1025     from src.reporting import emit_delivery_report
1026     duration_s = (stop_frame / fps) if (stop_frame and fps) else None
1027     report_paths = emit_delivery_report(
1028         out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
1029         video_path=args.video_path, video_duration_s=duration_s,
1030         source_format=("cvat" if args.input else "csv"), step=step,
1031     )
1032
1033     print(md)

```

17. assistant (2026-06-19T16:40:42.758Z)

I have everything I need. Let me synthesize the verdict.

Key findings:

1. ****xml-finding logic is accurately described**** (lines 211-216): zip path picks ``endswith("annotations.xml")`` first, else ``endswith(".xml")``, else raises ``ValueError``.
2. ****The "silent continue" is real****: ``load_canonical_csv`` has two ``continue`` statements (line 580 for ``rn`` is ``None``, line 589 for missing start/end) with no logging. There is NO logger in the module ? the entire ``src/`` tree of the collector uses no ``logging`/`getLogger`` at all. The module does NOT import ``logging``.
3. ****Tests lock in silent-skip behavior****: ``test_empty_cvat_converts_to_zero_records``, ``test_load_canonical_csv_decimal_and_mmss``, ``test_load_canonical_csv_keeps_rally_starting_at_zero`` ? all assert counts/values, none assert silence (no ``caplog`` usage). A warning wouldn't break them.
4. ****There IS broader observability**** the claim omits: at CLI level (``validate_delivery``), zero-records is loudly gated (``sys.exit(2)`/`^(1)``), and there's an ``emit_delivery_report`` observability layer. The per-row silent drop the claim describes is real but the "import gate" framing is partly mitigated by the all-or-nothing zero-records gate.

18. user (2026-06-19T16:41:15.573Z)

Structured output provided successfully

19. assistant (2026-06-19T16:41:23.070Z)

I've completed the adversarial review of claim C-cvat-parser. Verdict: ****partially-accurate****, low severity, not a real bug.

Summary of findings:

- The factual code descriptions are accurate (verified at `C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally\_cvat.py:211-216` for the zip xml-finding logic, lines 579-589 for the two silent `continue` skips, and `src\core\parsing.py:9-19` for safe_int).
- The "silent continue" is real, and there is NO logger in the module ? a repo-wide grep confirms the entire `src/` tree uses no `logging`/`getLogger`.
- The claim overstates the gate impact: a fully-empty CSV IS hard-gated by the CLI caller (`curate.py:1003-1005`, `sys.exit(1)`). Only a partial undercount slips through silently ? which is the actual gap.
- Existing tests don't assert silence (no caplog), so the proposed warning wouldn't regress anything.
- The proposed fix is safe but under-specified for the repo's no-logger convention; an Issue/summary entry would fit better than a bare `logger.warning`.
- Flagged the cross-repo coupling: this is a sports-data-collector change bundled into the indexer's plan, violating the indexer's ONE-PR-per-work-item rule.